

SPARCS Hospital Discharge Database

SQL code to test relationships before creating the final ER/EER diagram

Purpose

Run these queries on the imported raw table before creating normalized tables. The results help confirm primary keys, foreign keys, and one-to-many or one-to-one relationships.

1. Select the database and check the raw data

```
USE sparcs_db;

SELECT COUNT(*) AS total_rows
FROM raw_inpatient_discharges;

SELECT *
FROM raw_inpatient_discharges
LIMIT 10;
```

Create a row identifier if the imported table does not already have one:

```
ALTER TABLE raw_inpatient_discharges
ADD COLUMN raw_row_id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY FIRST;
```

2. Test the Hospital relationship

These queries check whether one hospital is connected to many discharge records and whether a facility identifier maps to only one facility name.

```
SELECT
    permanent_facility_id,
    facility_name,
    COUNT(*) AS discharge_count
FROM raw_inpatient_discharges
GROUP BY permanent_facility_id, facility_name
ORDER BY discharge_count DESC;

SELECT
    permanent_facility_id,
    COUNT(DISTINCT facility_name) AS facility_names
FROM raw_inpatient_discharges
GROUP BY permanent_facility_id
HAVING COUNT(DISTINCT facility_name) > 1;
```

Expected relationship: Hospital 1 to many Inpatient Discharges.

3. Test the DRG relationship

```
SELECT
    apr_drg_code,
    COUNT(*) AS discharge_count
FROM raw_inpatient_discharges
GROUP BY apr_drg_code
ORDER BY discharge_count DESC;

SELECT
    apr_drg_code,
    COUNT(DISTINCT apr_drg_description) AS descriptions
FROM raw_inpatient_discharges
GROUP BY apr_drg_code
HAVING COUNT(DISTINCT apr_drg_description) > 1;
```

If the second query returns no rows, one DRG code has one description and DRG can be a lookup table.

4. Test the MDC relationship

```
SELECT
    apr_mdc_code,
    COUNT(DISTINCT apr_mdc_description) AS descriptions
FROM raw_inpatient_discharges
GROUP BY apr_mdc_code
HAVING COUNT(DISTINCT apr_mdc_description) > 1;
```

5. Test Diagnosis and Procedure relationships

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(ccsr_diagnosis_code) AS rows_with_diagnosis
FROM raw_inpatient_discharges;

SELECT
  COUNT(*) AS total_rows,
  COUNT(ccsr_procedure_code) AS rows_with_procedure,
  COUNT(*) - COUNT(ccsr_procedure_code) AS rows_without_procedure
FROM raw_inpatient_discharges;
```

The procedure query shows whether every discharge has a procedure. A missing procedure value means the relationship is optional.

6. Test demographic, payment, and financial completeness

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(gender) AS gender_values,
  COUNT(race) AS race_values,
  COUNT(ethnicity) AS ethnicity_values
FROM raw_inpatient_discharges;

SELECT
  COUNT(*) AS total_rows,
  COUNT(payment_typology_1) AS primary_payment,
  COUNT(payment_typology_2) AS secondary_payment,
  COUNT(payment_typology_3) AS tertiary_payment
FROM raw_inpatient_discharges;

SELECT
  COUNT(*) AS total_rows,
  COUNT(total_charges) AS charge_values,
  COUNT(total_costs) AS cost_values
FROM raw_inpatient_discharges;
```

7. Relationship summary for the ER diagram

```
-- Use the query results to confirm these relationships:

Hospital          1 ----< Inpatient_Discharge
DRG               1 ----< Inpatient_Discharge
MDC               1 ----< Inpatient_Discharge
Inpatient_Discharge 1 ----1 Patient_Demographics
Inpatient_Discharge 1 ----0..1 Procedure
Inpatient_Discharge 1 ----1 Severity_Risk
Inpatient_Discharge 1 ----1 Payment
Inpatient_Discharge 1 ----1 Financial
```

Important note

Do not finalize the ER diagram only from these assumptions. Record the query results, explain the observed cardinalities, then create the normalized tables with primary keys and foreign keys. Finally, use MySQL Workbench Reverse Engineer to generate the logical EER diagram.

Suggested workflow: raw CSV table -> relationship tests -> normalized tables -> constraints -> Reverse Engineer diagram.